

• Easy - #88 - Merge Sorted Array

▲ Problem Statement: Given two sorted arrays "nums 1" & "nums 2" merge them into one sorted array stored inside "nums 1".

"nums 1" has enough extra space at the end to store all elements of "nums 2".

▲ Main Solution: Use three pointers starting from the end.

This avoids overwriting elements in "nums 1".

- "**i**" → last valid element in "**nums 1**"
- "**j**" → last element in "**nums 2**"
- "**k**" → last position in "**nums 1**"

Compare elements from the back & place the larger one at "**nums 1[k]**"

▲ Brute Force → Copy all elements of "**nums 2**" into the empty positions of "**nums 1**", then sort the entire array.

▲ Algorithm 1 → Set :- $i = m - 1;$
 $j = n - 1;$
 $k = m + n - 1;$

2 → While both arrays still have elements:-

- Compare "**nums 1[i]**" & "**nums 2[j]**"
- Put the larger element at "**nums 1[k]**"
- Move the corresponding pointer backward.
- Move "**k**" backward.

3 → If elements remain in "**nums 2**", copy them into "**nums 1**"

4 → No need to copy remaining elements from "**nums 1**" because they are already in the correct position.

▲ Important Points → • Start merging from the end to avoid overwriting values in "**nums 1**".

- "i" points to valid elements of "nums1"
- "j" points to elements of "nums2"
- "k" points to the position where the next largest elements goes.
- Remaining elements of "nums1" do not need to be copied.
- Only remaining elements of "nums2" need to be copied.

▲ Complexity → Brute Force → $O((m+n) \log(m+n))$ & $O(1)$
 Three Pointers → $O(m+n)$ & $O(1)$